ОТЧЁТ О ПРОВЕДЕНИИ РАБОТ

по групповому проекту

«Учёт расходов семьи»

Вариант 4. Разработка приложения в Visual Studio + Git

Дисциплина: МДК 05.02

Группа: Исп 31-9

Состав группы:

- Ильиных Илья — Тимлид / разработчик
- Галичанин Денис — Бэкенд разработчик / тестировщик
- Столбов Лев — Создание диаграмм в draw.io и документация
- Русских Роман — Ответственный за документацию и презентации

Преподаватель: Шадрина.А.Ю

Ссылка на репозиторий: [ссылка на GitHub]

Ижевск, 2026 г.

1. Цель и задачи проекта

Цель проекта — разработать настольное приложение на Windows Forms для учёта доходов и расходов семьи и освоить командную работу в Visual Studio с использованием Git: ветки, pull request’ы, код-ревью.

Задачи:

- спроектировать модель данных и архитектуру приложения, подготовить четыре диаграммы (use case, ER, классы/компоненты, последовательность);
- реализовать учёт операций (добавление, изменение, удаление), справочники участников и категорий, фильтрацию, итоги и отчёт;
- написать модульные тесты и запускать их в обозревателе тестов Visual Studio;
- подготовить README, руководство пользователя, отчёт и презентацию;
- вести разработку в Git: репозиторий с первого дня, осмысленные коммиты, ветка на каждую фичу.

2. Распределение ролей и обязанностей

Участник	**Роль**	**Что делал**	**Коммитов / PR**
Ильиных И.	Тимлид / разработчик	Планирование, архитектура, ядро (БД, репозиторий, отчёты), интерфейс WinForms, ревью	7 PR 3
Галичанин Д.	Бэкенд разработчик / тестировщик	Создание репозитория и решения, .gitignore, защита main, настройка веток и PR, слияния, тесты	12 PR 3
Столбов Л.	Создание диаграмм в draw.io и документация	Диаграммы use case, ER, классов/архитектуры, последовательности; помощь с отчётом и руководством	6
Русских Р.	Документация и презентации	README, отчёт, руководство пользователя, презентация	10

3. Хронология работ

Этап	**Что сделано**	**Трудности**
1. Выбор темы и планирование	Выбран вариант 4 «Учёт расходов семьи», распределены роли, создан репозиторий, согласованы правила веток и коммитов (ветка на каждую фичу, слияние через pull request).	Не сразу распределили задачи; на первой встрече закрепили роли и порядок работы.
2. Проектирование	Определены сущности (участники, категории, операции), схема БД,	Спорили, как связать операции с категориями и

Этап	Что сделано	Трудности
	архитектура приложения; построены 4 диаграммы.	участниками; остановились на трёх связанных таблицах.
3. Разработка	Создана база данных Access (таблицы участников, категорий и операций со связями), слой доступа к данным, расчёт итогов и проверка вводимых данных. Интерфейс: главная форма, форма ввода операции, справочники, отчёт.	Ошибка подключения к базе Access из-за разрядности драйвера; сборку перевели на платформу x64.
4. Тестирование	Модульные тесты xUnit (доступ к данным, отчёты, проверка вводимых данных), ручная проверка сценариев в приложении.	Часть тестов падала из-за пустой базы; добавили подготовку тестовых данных.
5. Документирование	README.md, руководство пользователя, отчёт, презентация.	Версии документов у участников расходились; общие файлы стали хранить в репозитории.
6. Защита	Подготовка живого демо, репетиция выступления.	Не хватало времени на репетицию; заранее распределили, кто что показывает.

4. Использованные технологии и инструменты

- Visual Studio — решение из трёх проектов, отладчик, Git-интеграция, обозреватель тестов, менеджер NuGet;
- C# фрейм ворк, Windows Forms — интерфейс;
- Microsoft Access — хранение данных в одном файле, без установки сервера БД;
- xUnit — модульные тесты;
- Git и GitHub — репозиторий, ветки, pull request’ы, код-ревью;
- StarUML, Draw.io / диаграммы — схемы для документации.

5. Возникшие проблемы и способы их решения

Технические решения, принятые на этапе проектирования:

- Выбрана СУБД Microsoft Access (файл .accdb): все данные хранятся в одном файле без установки сервера БД, а таблицы, связи и записи можно открыть и проверить прямо в Access. Подключение выполняется через OleDb (драйвер Microsoft Access Database Engine).
- Драйвер Access Database Engine должен совпадать по разрядности с приложением (x64 или x86), поэтому проект собирается под платформу x64, а на компьютере

устанавливается соответствующая версия драйвера; без этого при запуске возникает ошибка подключения к базе.

- Сумма в коде хранится как decimal, чтобы избежать ошибок округления при подсчёте итогов; в Access поле «Сумма» имеет тип «Денежный».
- В схеме данных Access между таблицами настроены связи с обеспечением целостности данных: категорию или участника, у которых есть операции, удалить нельзя; интерфейс показывает понятное сообщение вместо ошибки.
- Логика (валидация, отчёты, запросы к БД) вынесена в проект Core, чтобы её можно было тестировать без запуска форм.

Проблемы, с которыми столкнулась команда в ходе работы:

Проблема	**Как решили**
Ошибки в коде при разработке	Проверяли код в Visual Studio, находили ошибки с помощью отладчика и исправляли их
Не совпадала разрядность драйвера Access Database Engine и приложения (ошибка подключения к базе при запуске).	Сборку перевели на платформу x64 и установили драйвер x64 на все компьютеры команды.
Конфликт слияния в главной форме: два участника одновременно правили интерфейс.	Конфликт разобрали вместе в Visual Studio, оставили обе правки; затем разделили формы между разработчиками.

6. Выводы

В результате создано работающее приложение «Учёт расходов семьи»: учёт доходов и расходов, справочники, фильтры, итоги и отчёт; логика покрыта модульными тестами. Работа велась в Git по схеме «ветка на фичу — pull request — ревью».

Не успели реализовать графики в отчёте, импорт данных и полноценные профили пользователей и разграничение доступа; оставили их как направления развития.

Научились работать в одном репозитории: создавать ветки под каждую задачу, оформлять pull request’ы и проверять код друг друга. Разделение проекта на данные, логику и интерфейс упростило совместную работу и тестирование. В Visual Studio освоили отладчик, менеджер NuGet и обозреватель тестов.

Что можно улучшить: бюджеты и лимиты по категориям, диаграммы в отчёте, импорт данных.